دليل المطوّرين

البحيري CRM
طريقة العمل والنشر

كيف نطوّر ونجرّب وننشر بأمان بين مطوّرَين: بيئة تجريبية **dev** وبيئة معتمدة **main**، وكل `push` يُنشر تلقائيًا على الرابط الخاص به.

```
main https://elbehiry.engsaber.space
```

```
dev https://dev-elbehiry.engsaber.space
```

```
repo github.com/SaberGD/elbehiry-crm
```

1 الفكرة في دقيقة

main — الإنتاج

النسخة المعتمدة التي يراها الفريق والعملاء. لا نكتب عليها مباشرة أبدًا.

`elbehiry.engsaber.space`

dev — التطوير

كل شغل جديد يبدأ هنا ويُجرَّب قبل أي اعتماد.

`dev-elbehiry.engsaber.space`



القواعد الأربع

1. **لا push مباشر على main.** الطريق الوحيد إليها Pull Request من dev.
2. **لا force-push** على dev أو main (`git push -f` ممنوع).
3. **جرّب على رابط dev** واتفقوا إن التعديل تمام قبل الـ Merge.
4. **commit صغير وواضح**، وفي كل مرة اسحب آخر نسخة قبل ما تبدأ.

النشر تلقائي: أي push على dev يُنشر على رابط dev، وأي merge على main يُنشر على رابط الإنتاج. لا حاجة لرفع ملفات يدويًا.

2 الشغل اليومي خطوة بخطوة

أ) البداية (مرة واحدة)

```
git clone https://github.com/SaberGD/elbehiry-crm.git
cd elbehiry-crm
bun install --frozen-lockfile
bun run dev            # يفتح على http://localhost:3000
```

ب) قبل أي شغل: حدّث dev

```
git checkout dev
git pull origin dev
```

ج) اعمل التعديل واحفظه

```
bun run lint && bun run test     # لازم يعدّوا قبل ما ترفع
git add -A
git commit -m "feat: add after-sale follow-up"
git push origin dev            # ← ينشر تلقائيًا على dev-elbehiry
```

> **لو معاك مطوّر تاني على dev في نفس الوقت:** اعمل `git pull --rebase origin dev` قبل الـ push. أو اشتغل كل واحد على فرع قصير: `git checkout -b feature/اسم-الميزة` ثم ادفعه وافتح PR إلى dev.

د) اعتماد التعديل على main

1. افتح الموقع التجريبي `dev-elbehiry.engsaber.space` وجرّب التعديل بنفسك.
2. اتفقوا (أنت والمطوّر التاني) إن كل شيء تمام.
3. في GitHub: **Pull requests** ← **New** من `dev` إلى `main`.
4. انتظر ✅ من فحوصات الـ Actions (lint / test / build)، ثم **Merge** بنوع *Merge commit*.
5. خلال دقيقة أو اثنتين يظهر التغيير على `elbehiry.engsaber.space`.

هـ) بعد الـ Merge: خلّي dev تلحق main

```
git checkout dev
git pull origin main           # يجيب أي تغييرات دخلت main
git push origin dev
```

3 رسائل الـ commit والمراجعة

البادئة	تُستخدم لـ	مثال
`feat:`	ميزة جديدة	`feat: متابعة بعد التسليم`
`fix:`	إصلاح خطأ	`fix: تاريخ المتابعة التالية`
`docs:`	توثيق فقط	`docs: تحديث الدليل`
`refactor:`	تنظيف بدون تغيير سلوك	`refactor: فصل hook المزامنة`
`ci:`	إعدادات النشر والفحص	`ci: تحديث الـ workflow`

قبل فتح Pull Request إلى main

- الـ Actions على dev **أخضر** (lint, test, build).
- جرّبت الميزة على رابط dev بحساب من كل دور تأثّر بها (مدير، مشرف، موظف).
- لم تُغيّر `firestore.rules` إلا بعد الاتفاق، ونُشرت في Firebase Console يدويًا.
- وصف الـ PR: ماذا تغيّر ولماذا وكيف جرّبته.

حل التعارضات (Conflicts)

```
git checkout dev
git pull origin main          # لو ظهر تعارض، افتح الملفات وابحث عن <<<<<<<
# اختر الصح، ثم:
git add -A
git commit
bun run lint && bun run test && git push origin dev
```

> **لا تحلّ التعارض بمسح تعديل زميلك.** لو مش متأكد، كلّمه قبل ما تحسم.

الرجوع عن تعديل وصل main

```
git checkout main && git pull origin main
git revert -m 1 <MERGE_COMMIT_SHA>      # عكسي آمن يعمل commit
# افتح PR بالتغيير ده أو ادفعه بالاتفاق
```

4 الإعدادات والمخاطر

إعداد لمرة واحدة (صاحب الريبو)

الإعداد	أين
`FTP_SERVER` و `FTP_USERNAME` و `FTP_PASSWORD`	GitHub. بيانات حساب FTP من hPanel. ← Settings ← Secrets and variables ← Actions
حماية فرع `main`	Settings ← Branches ← Add rule: *Require a pull request* + *Require status checks (build)* + منع force-push.
صلاحية المطوّر الثاني	Settings ← Collaborators ← Add (صلاحية Write).
مسار كل موقع على الاستضافة	`.github/workflows/deploy.yml` ← `SERVER_DIR_MAIN` و `SERVER_DIR_DEV`.

Firebase وقاعدة البيانات

انتبه: موقع dev وموقع main يتصلان حاليًا بنفس مشروع Firebase (`albehairy-group`). أي عميل تضيفه من dev يظهر في الإنتاج. استخدم بيانات تجريبية واضحة على dev، أو أنشئ مشروع Firebase منفصلًا للتطوير (الأفضل).

- قواعد Firestore (`firestore.rules`) **لا تُنشر تلقائيًا**؛ تُلصق في Rules ← Firestore ← Firebase Console.
- تسجيل الدخول: Email/Password مفعّل في Firebase Authentication. أول مستخدم يعمل "إعداد حساب المدير".
- الأدوار تُخزَّن في `authProfiles/{uid}` ولا يُعدّلها إلا مدير النظام.
- لا تضع أي مفاتيح سرية في الكود. الإعداد العام لـ Firebase مقبول، والحماية تأتي من القواعد.

لو فشل النشر

العَرَض	الحل
الـ Action أحمر في *lint* أو *test*	شغّل `bun run lint` و `bun run test` محليًا وصلّح الخطأ.
فشل في خطوة FTP	راجع الـ Secrets ومسار `server-dir`.
الموقع لم يتغير بعد النشر	اعمل تحديث قوي (Ctrl+Shift+R). التطبيق PWA، وقد يعرض رسالة "نسخة جديدة متاحة".